

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Домашнее задание №5

Выполнила:

Шакина Анна

Группа

J3211

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

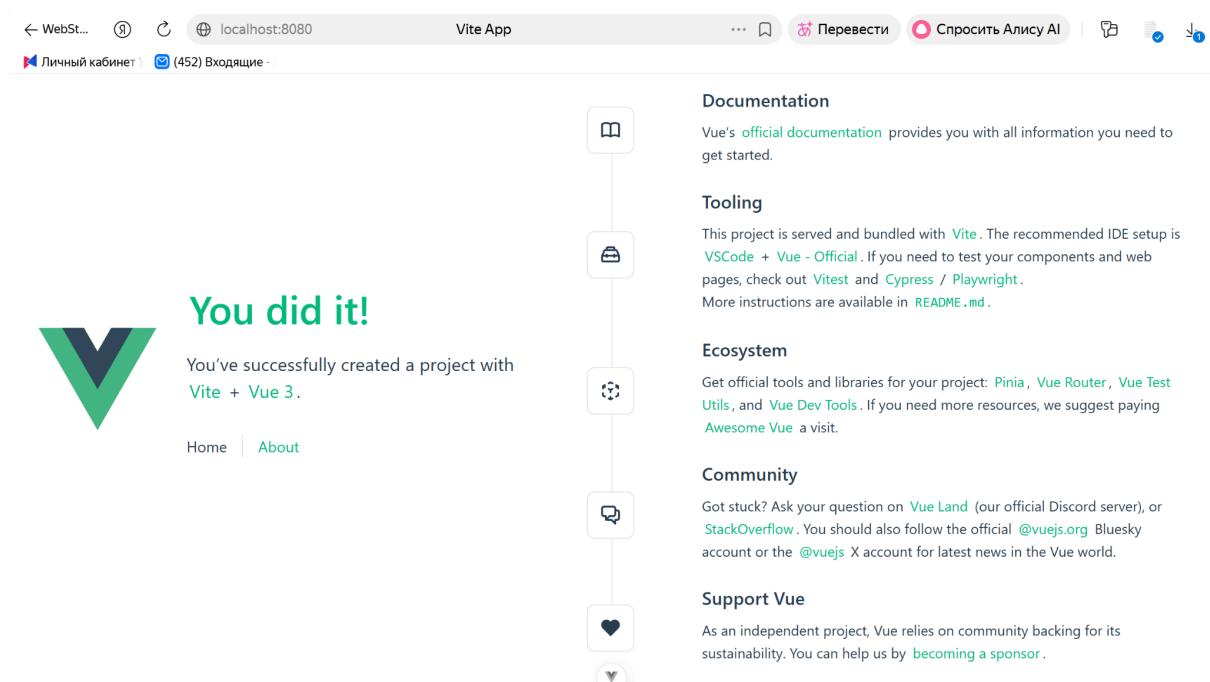
Задача

Перенести существующее веб-приложение «Мероприятия» с JavaScript на фреймворк Vue.js.

Ход работы

1. Инициализация проекта

Создала Vue-проект через `npm init vue@latest`.



2. Настройка API-слоя

Создала `instance.js`. Разделила API на классы – `EventsApi`, `UsersApi`, `RegistrationsApi`. В `index.js` создала IoC-контейнер.

```

1  import axios from 'axios'
2
3  const instance : AxiosInstance = axios.create({
4    baseURL: 'http://localhost:3000'
5  })
6
7  instance.interceptors.request.use( onFulfilled: (config) => {
8    const token : string = localStorage.getItem( key: 'accessToken')
9    if (token) {
10      config.headers.Authorization = `Bearer ${token}`
11    }
12    return config
13  })
14
15  export default instance Show usages

```

```

1  import instance from '@api/instance'
2  import EventsApi from '@api/events'
3  import UsersApi from '@api/users'
4  import RegistrationsApi from '@api/registrations'
5
6  export const eventsApi : EventsApi = new EventsApi(instance) Show usages
7  export const usersApi : UsersApi = new UsersApi(instance) Show usages
8  export const registrationsApi : RegistrationsApi = new RegistrationsApi(instance) Show usages

```

3. Настройка Pinia

Создала stores/index.js с pinia-persists, хранилище events.js для мероприятий и auth.js для авторизации.

```

1  import { defineStore } from 'pinia'
2  import { eventsApi } from '@api'
3
4  const useEventsStore : StoreDefinition<'events', {...}, {...}, {...}> = defineStore( id: 'events', options: {
5    state: () : {events: []} => ({ events: [] }),
6    actions: {
7      async loadEvents() : Promise<void> {
8        const response = await eventsApi.getAll()
9        this.events = response.data
10      }
11    }
12  })
13
14  export default useEventsStore Show usages

```

4. Настройка роутера

Настроила маршруты для всех страниц с защитой через beforeEach.

```

1 > import ...
3
4 const router : Router = createRouter( options: {
5   history: createWebHistory(import.meta.env.BASE_URL),
6   routes: [
7     { path: '/', name: 'home', component: () : Promise<...> => import('@views/HomePage.vue') },
8     { path: '/event/:id', name: 'event', component: () : Promise<...> => import('@views/EventPage.vue') },
9     { path: '/login', name: 'login', component: () : Promise<...> => import('@views/LoginPage.vue') },
10    { path: '/profile', name: 'profile', component: () : Promise<...> => import('@views/ProfilePage.vue') },
11    { path: '/teacher', name: 'teacher', component: () : Promise<...> => import('@views/TeacherPage.vue') },
12    { path: '/admin', name: 'admin', component: () : Promise<...> => import('@views/AdminPage.vue') },
13  ]
14 })
15
16 // Защита роутов
17 router.beforeEach( guard: (to : RouteLocationNormalized ) : { ... } | undefined => {
18   const auth : Store<"auth", (...), (...), (...)> = useAuthStore()
19   if (to.meta.requiresAuth && !auth.isLoggedIn) return { name: 'login' }
20   if (to.meta.role === 'admin' && !auth.isAdmin) return { name: 'home' }
21   if (to.meta.role === 'teacher' && !auth.isTeacher) return { name: 'home' }
22 })
23
24 export default router

```

5. Компонент EventCard.vue

Создала переиспользуемый компонент карточки мероприятия.
Используются props для передачи данных, метод formatDate.

```

1 <template> Show component usages
2 <div class="col">
3   <div class="card event-card-custom h-100">
4     <div class="event-card-image-wrap">
5       
8     </div>
9     <div class="event-card-body">
10      <h5 class="event-card-title">{{ event.title }}</h5>
11      <p class="event-card-text">{{ event.description }}</p>
12      <div class="event-card-meta">
13        <span class="event-meta-pill">{{ formatDate(event.date) }}</span>
14        <span class="event-meta-pill">{{ event.place }}</span>
15      </div>
16      <router-link :to="/event/${event.id}" class="site-button mt-2">
17        Подробнее
18      </router-link>
19    </div>
20  </div>
21 </template>

```

```

23
24 <script>
25 export default { Show usages
26   name: 'EventCard',
27   props: {
28     event: {
29       type: Object,
30       required: true
31     }
32   },
33   methods: {
34     formatDate(date) {
35       if (!date) return 'Дата не указана'
36       return new Date(date).toLocaleDateString( {locales: 'ru-RU'} )
37     }
38   }
39 }
40 </script>

```

6. BaseLayout.vue

```
BaseLayout.vue x
1 <template> Show component usages
2   <div>
3     <app-navbar />
4     <main class="main pt-5">
5       <slot />
6     </main>
7     <app-footer />
8   </div>
9 </template>
10
11 <script>
12 import AppNavbar from '@components/AppNavbar.vue'
13 import AppFooter from '@components/AppFooter.vue'
14
15 export default { Show usages
16   name: 'BaseLayout',
17   components: { AppNavbar, AppFooter }
18 }
19 </script>
```

7. Главная страница HomePage.vue

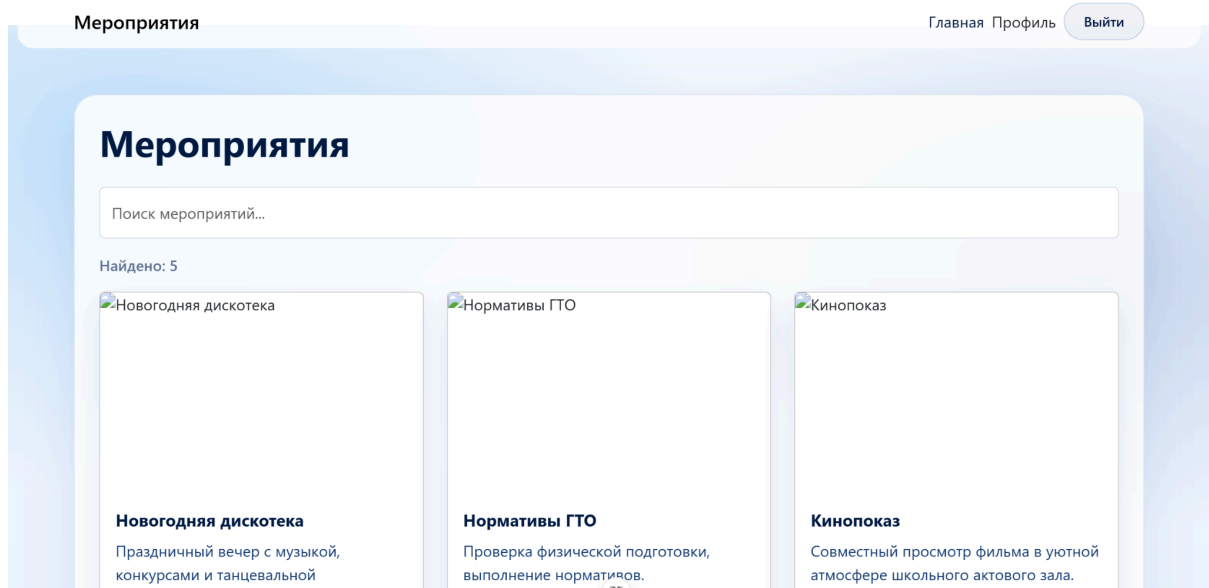
Используется v-for для вывода списка мероприятий, v-model для поиска, mounted для загрузки данных, computed для фильтрации.

```
HomePage.vue x
1 <template> Show component usages
2 <base-layout>
3   <section class="events-explorer py-5">
4     <div class="container pt-4">
5       <div class="events-shell">
6         <div class="events-intro">
7           <h1 class="events-title">Мероприятия</h1>
8         </div>
9
10        <!-- Поиск -->
11        <input
12          v-model="search"
13          type="text"
14          class="search-input form-control mb-3"
15          placeholder="Поиск мероприятий...">
16
17        <p class="events-counter">Найдено: {{ filteredEvents.length }}</p>
18
19        <!-- Список мероприятий -->
20        <div v-if="filteredEvents.length" class="row row-cols-1 row-cols-md-2 row-cols-lg-3 g-4">
21          <event-card
22            v-for="event in filteredEvents"
23            :key="event.id"
24            :event="event" />
25        </div>
26
27        <div v-else class="events-empty">
28          <p class="events-empty-text">Мероприятия не найдены.</p>
29        </div>
30      </div>
31    </div>
32  </section>
33 </base-layout>
34 </template>
35
36 <script>
37 import { mapState, mapActions } from 'pinia'
38 import useEventsStore from '@stores/events'
39 import BaseLayout from '@layouts/BaseLayout.vue'
40 import EventCard from '@components/EventCard.vue'
```

```

42 export default { Show usages
43   name: 'HomePage',
44   components: { BaseLayout, EventCard },
45   data() {
46     return {
47       search: ''
48     }
49   },
50   computed: {
51     ...mapState(useEventsStore, { keys: ['events']}),
52     filteredEvents() {
53       if (!this.search.trim()) return this.events
54       const q = this.search.toLowerCase()
55       return this.events.filter(e =>
56         e.title?.toLowerCase().includes(q) ||
57         e.description?.toLowerCase().includes(q)
58       )
59     }
60   },
61   methods: {
62     ...mapActions(useEventsStore, { keyMapper: ['loadEvents']})
63   },
64   mounted() {
65     this.loadEvents()
66   }
67 }
68 </script>

```



8. Страница авторизации LoginPage.vue

Форма с v-model для полей email и пароля.

```

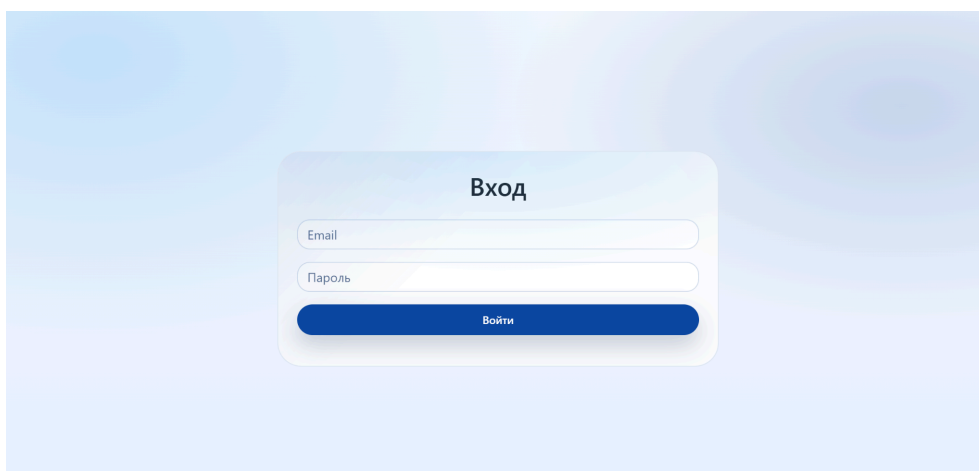
1 <template> Show component usages
2 <div class="auth-page d-flex align-items-center justify-content-center min-vh-100">
3   <div class="auth-shell w-100">
4     <div class="auth-card p-4">
5       <h2 class="mb-4 text-center">Вход</h2>
6       <div v-if="error" class="auth-form-msg auth-form-msg-error mb-3">{{ error }}</div>
7       <div class="d-flex flex-column gap-3">
8         <input
9           v-model="form.email"
10          type="email"
11          class="form-control"
12          placeholder="Email">
13         <input
14           v-model="form.password"
15           type="password"
16           class="form-control"
17           placeholder="Пароль">
18         <button class="auth-btn" @click="handleLogin">Войти</button>
19       </div>
20     </div>
21   </div>
22 </div>
23 </template>

```

```

26 <script>
27 import { mapActions } from 'pinia'
28 import useAuthStore from '@stores/auth'
29 export default { Show usages
30   name: 'LoginPage',
31   data() {
32     return {
33       form: { email: '', password: '' },
34       error: ''
35     }
36   },
37   methods: {
38     ...mapActions(useAuthStore, { keyMapper: ['login'] }),
39     async handleLogin() {
40       try {
41         this.error = ''
42         await this.login(this.form.email, this.form.password)
43         this.$router.push('/')
44       } catch {
45         this.error = 'Неверный email или пароль'
46       }
47     }
48   }
49 }
50 </script>

```



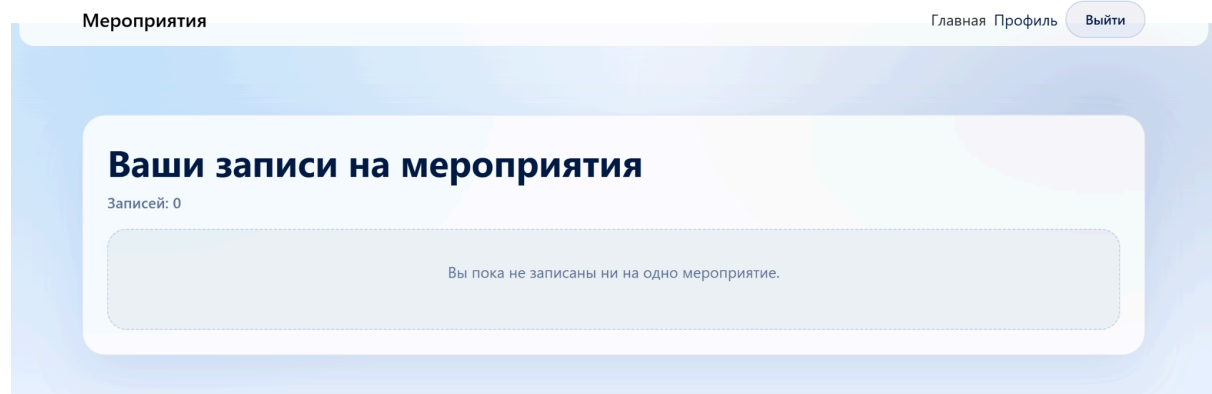
9. Страница профиля ProfilePage.vue

Используется v-if / v-else-if / v-else для отображения разных состояний (не вошёл / нет записей / список записей)

```
▼ HomePage.vue  ▼ LoginPage.vue  ▼ ProfilePage.vue x
1  <template> Show component usages
2  <base-layout>
3    <section class="events-explorer py-5">
4      <div class="container pt-4 pt-lg-5">
5        <div class="events-shell">
6          <h1 class="events-title">Ваши записи на мероприятия</h1>
7          <p class="events-counter">Записей: {{ myRegistrations.length }}</p>
8
9          <div v-if="!isLoggedIn" class="profile-state-card">
10             <p>Войдите, чтобы увидеть свои записи.</p>
11             <router-link to="/login" class="site-button">Войти</router-link>
12          </div>
13
14          <div v-else-if="myRegistrations.length === 0" class="events-empty">
15             <p class="events-empty-text">Вы пока не записаны ни на одно мероприятие.</p>
16          </div>
17
18          <div v-else class="row row-cols-1 row-cols-md-2 g-4">
19             <event-card
20               v-for="reg in myRegistrations"
21               :key="reg.id"
22               :event="getEventById(reg.eventId)" />
23             </div>
24          </div>
25        </div>
26      </section>
27    </base-layout>
28  </template>
```

```
30  <script>
31  import { mapState, mapActions } from 'pinia'
32  import useAuthStore from '@stores/auth'
33  import useEventsStore from '@stores/events'
34  import { registrationsApi } from '@api'
35  import BaseLayout from '@layouts/BaseLayout.vue'
36  import EventCard from '@components/EventCard.vue'
37
38  export default { Show usages
39    name: 'ProfilePage',
40    components: { BaseLayout, EventCard },
41    data() {
42      return { registrations: [] }
43    },
44    computed: {
45      ...mapState(useAuthStore, keys: ['user', 'isLoggedIn']),
46      ...mapState(useEventsStore, keys: ['events']),
47      myRegistrations() {
48        return this.registrations.filter(r =>
49          String(r.userId) === String(this.user?.id) && r.status === 'active'
50        )
51      }
52    },
53    methods: {
54      ...mapActions(useEventsStore, keyMapper: ['loadEvents']),
55      getEventById(id) {
56        return this.events.find(e => String(e.id) === String(id)) || {}
```

```
58      async loadRegistrations() {
59        const response = await registrationsApi.getAll()
60        this.registrations = response.data
61      }
62    },
63    async mounted() {
64      await this.loadEvents()
65      if (this.isLoggedIn) await this.loadRegistrations()
66    }
67  }
68  </script>
```

Вывод

В ходе выполнения домашнего задания был создан Vue-проект на основе существующего приложения для управления мероприятиями. Проект был разделён на логические слои: API-слой (src/api/), хранилища состояния (src/stores/), лейауты, компоненты и представления. Были применены ключевые возможности Vue.js: директива v-for для вывода списка мероприятий, v-model для двустороннего связывания данных в форме поиска и авторизации, v-if / v-else-if / v-else для условного рендеринга, props для передачи данных в дочерние компоненты, mounted для загрузки данных при открытии страницы. Настроен Vue Router с защитой маршрутов и Pinia для управления глобальным состоянием с сохранением в localStorage.